

sppComp

nicolo-tellini

2023-02-03



© logo N.T. & PhD Chiara Vischioni

Software:

- *GNU Awk 5.0.0 or higher (gawk) [Ubuntu: apt-get install gawk]*
- *bwa [Ubuntu: apt-get install bwa]*
- *samtools v.1.10 or higher [Ubuntu: apt-get install samtools]*

R libraries :

- *Biostrings (version used 2.66.0)*
- *GenomicRanges (version used 1.50.2)*
- *data.table (version used 1.14.6)*
- *ggplot2 (version used 3.4.0)*

USER'S SETTINGS

```
rm(list = ls()) # clean GlobalEnv

# START USER'S SETTINGS ----
binsize=10000
# parallel runs
nSamples=5 # number of samples
nThreads=4 # per-sample number of threads
# END USER'S SETTINGS ----

Sys.setenv(nSamples = nSamples)
Sys.setenv(nThreads = nThreads)

library(rstudioapi)
library(Biostrings)
```

```

## Loading required package: BiocGenerics

##
## Attaching package: 'BiocGenerics'

## The following objects are masked from 'package:stats':
##
##     IQR, mad, sd, var, xtabs

## The following objects are masked from 'package:base':
##
##     anyDuplicated, aperm, append, as.data.frame, basename, cbind,
##     colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
##     get, grep, grepl, intersect, is.unsorted, lapply, Map, mapply,
##     match, mget, order, paste, pmax, pmax.int, pmin, pmin.int,
##     Position, rank, rbind, Reduce, rownames, sapply, setdiff, sort,
##     table, tapply, union, unique, unsplit, which.max, which.min

## Loading required package: S4Vectors

## Loading required package: stats4

##
## Attaching package: 'S4Vectors'

## The following objects are masked from 'package:base':
##
##     expand.grid, I, unname

## Loading required package: IRanges

## Loading required package: XVector

## Loading required package: GenomeInfoDb

##
## Attaching package: 'Biostrings'

## The following object is masked from 'package:base':
##
##     strsplit

library(GenomicRanges)
library(data.table)

##
## Attaching package: 'data.table'

## The following object is masked from 'package:GenomicRanges':
##
##     shift

```

```
## The following object is masked from 'package:IRanges':
##
##     shift
```

```
## The following objects are masked from 'package:S4Vectors':
##
##     first, second
```

```
library(ggplot2)
library(tidytree)
```

```
## If you use the ggtree package suite in published research, please cite
## the appropriate paper(s):
##
## G Yu. Data Integration, Manipulation and Visualization of Phylogenetic
## Trees (1st ed.). Chapman and Hall/CRC. 2022. ISBN: 9781032233574
##
## Guangchuang Yu. Using ggtree to visualize data on tree-like structures.
## Current Protocols in Bioinformatics. 2020, 69:e96. doi:10.1002/cpbi.96
```

```
##
## Attaching package: 'tidytree'
```

```
## The following object is masked from 'package:S4Vectors':
##
##     rename
```

```
## The following object is masked from 'package:stats':
##
##     filter
```

```
BaseDir <- dirname(rstudioapi::getSourceEditorContext())$path)
```

step 1: binning multi-species FASTA

```
# Mon Feb 21 16:53:01 2022

# Title: FASTA binner
# Author: Nicolò T.
# Status: Complete
# Input: /rep/saccharomyces_all_assemblies.fa
# Output: A BED file with bins in ./rep dir

# Comment:
# 1) required libraries:
# Biostrings (2.66.0), GenomicRanges (1.50.2) and data.table (1.14.6).
# 3) library v.s correspond to those used during the development of the pipeline

# Options ---

options(stringsAsFactors = F,width = 60,warn = 1)
```

```

setwd(BaseDir)

# body ----

# read FASTA genome
mygenome <- readDNASTringSet(
  paste0(BaseDir, "/rep/saccharomyces_all_assemblies.fa")
)

# get chr sizes
chrSizes <- width(mygenome)

# name chrSizes
names(chrSizes) <- names(mygenome)

# generate the bins
bins <- as.data.frame(tileGenome(
  chrSizes, tilewidth=binsize, cut.last.tile.in.chrom=T)
)

# remove strand and width columns
bins$strand <- NULL

bins$width <- NULL

bins <- bins[!is.na(bins[,1]),]

# save the table as BED file in ./rep
fwrite(x = bins, file = paste0(BaseDir, "/rep/binnedGenome.", binsize, ".bed"),
  append = F, quote = F, sep = "\t", row.names = F, col.names = F)

print("STEP1: completed!")

```

step 2: indexing, mapping and coverage extraction

```

#!/bin/bash

# base folder
BaseDir=$(pwd)

# check logs folder
if [[ ! -d ${BaseDir}/logs ]]; then mkdir ${BaseDir}/logs; fi

# enter base folder
cd $BaseDir

# directory with mapping data
mapDir=$BaseDir/map

if [[ ! -d $mapDir ]]; then
mkdir $mapDir
fi

```

```

# directory for TXT Checkpoint (cps)
cpsDir=$BaseDir/cps
if [[ ! -d $cpsDir ]]; then
mkdir $cpsDir
fi

# TXT Checkpoint
FILE=$BaseDir/cps/cps.txt
if test -f "$FILE"; then
echo "$FILE exists"
else
touch $FILE
echo "$FILE created."
fi

# directory with coverage info
covDir=$BaseDir/cov
if [[ ! -d $covDir ]]; then
mkdir $covDir
fi

# add variable is (nThreads - 1)
# note: some commands add threads to the default value (1 thread)
add=$((nThreads - 1))

# index the concatenated assemblies in rep
bwa index ${BaseDir}/rep/*fa >/dev/null 2> ${BaseDir}/logs/Time.index2.err

for IndS in $(ls -Sr ./seq/*_1.*gz | cut -d"/" -f3 | cut -d"_" -f1)
do
# take the sample name.
# check if it is already stored in cps/cps.txt
cps=$(grep -w $IndS $BaseDir/cps/cps.txt)

# if $cps is empty the sample will be processed, skipped otherwise.
if [ -z "$cps" ]
then

# i increases progressively
((i++))
(
# check for the presence of paired files
revers=$(ls ./seq/$IndS"_2.*"*.gz)

# path to the assembly
IndR=$(ls ${BaseDir}/rep/*fa)

# ID assembly
refID=$(basename $IndR | cut -d"." -f 1)

# run BWA according to the fact the reads are either single or paired end
if [ ! -z "$revers" ]
then

```

```

bwa mem -t $nThreads $BaseDir/rep/$refID.fa $BaseDir/seq/$IndS"_1"*.gz" \
$BaseDir/seq/$IndS"_2"*.gz" -o $mapDir/$IndS".sam" 2> /dev/null

else

bwa mem -t $nThreads $BaseDir/rep/$refID.fa $BaseDir/seq/$IndS"_1"*.gz" \
-o $mapDir/$IndS".sam" 2> /dev/null

fi

# samtools workflow (http://www.htslib.org/workflow/fastq.html)
samtools fixmate -@ $add -O bam,level=1 -m $mapDir/$IndS".sam" \
$mapDir/$IndS".fix.bam"

samtools sort -l 1 -@ $nThreads $mapDir/$IndS".fix.bam" \
-T $BaseDir/tmp/$IndS".tmp.bam" -o $mapDir/$IndS".fix.srt.bam"

samtools markdup -@ $add -O bam,level=1 $mapDir/$IndS".fix.srt.bam" \
$mapDir/$IndS".fix.srt.mrk.bam"

samtools view -@ $add $mapDir/$IndS".fix.srt.mrk.bam" \
-o $mapDir/$IndS.$refID.bam

samtools index $mapDir/$IndS.$refID.bam

# comment the line below if you want to keep intermediate files
rm -r $mapDir/$IndS.sam $mapDir/$IndS.fix.bam $mapDir/$IndS.fix.srt.bam \
$mapDir/$IndS.fix.srt.mrk.bam
echo "mapping ${IndS} done!"
# samtools coverage does not accept BED file as input.
# The while loop below is a turn around.
while read -r line
do

chr=$(echo $line | cut -d" " -f1)
start=$(echo $line | cut -d" " -f2)
end=$(echo $line | cut -d" " -f3)
# measure coverage statistics across windows.
# Only reads with MQ >= 5 and BQ >= 20 are included.
# unmapped, secondary, QC failed and duplicated reads are excluded.
samtools coverage -q5 -Q20 --ff UNMAP,SECONDARY,QCFAIL,DUP \
-r $chr:$start-$end $mapDir/$IndS.$refID.bam | \
grep -v "#" >> $BaseDir/cov/$IndS".binned.cov"

done < $BaseDir"/rep/"*".bed"
echo "coverage ${IndS} done!"
wait
# add a column to binned.cov with the strain name
gawk -i inplace -v var="$IndS" -F'\t' 'BEGIN{OFS=FS}{print $0 OFS var}' \
$BaseDir/cov/$IndS".binned.cov"

# append strain name to ./cps/cps.txt
echo $IndS >> $BaseDir/cps/cps.txt

```

```

### uncomment the line below if you want to remove the BAMs and indexes
### rm $mapDir/$IndS.$refID.bam $mapDir/$IndS.$refID.bam.bai

)&

# The terminal part of the code hosts a job controller
# This is obtained by an if statement that permits the for loop
# to proceed to the next sample as soon as the number of running samples
# decreases, below the upper limit imposed by the variable $nSamples.
if (( i % nSamples == 0 )); then
  wait -n
  i=$((nSamples-1))
fi
done

wait

echo "STEP2: completed!"

```

step 3: plotting

```

# Mon Feb 21 18:03:18 2022

# Title: PLOT
# Author: Nicolò T.
# Status: Complete
# Input: /cov/binmed.cov files
# Output: plots

# Comment:
# 1) required libraries:
# ggplot2 (3.4.0), tidyverse (1.3.2) and data.table (1.14.6).
# 2) library v.s correspond to those used during the development of the pipeline

# SETTINGS ----

# Variables ----

refs <- c("Scer","Spar","Smik","Sjur","Skud","Sarb","Seub","Suva")

# note: rearranged chromosomes are assigned based on CEN.
# ex.: chrI of S.jur contain a long piece of chrXIII,
# nevertheless we refer to the former as chrI!
allChr <- c("chrI", "chrII", "chrIII", "chrIV", "chrV", "chrVI", "chrVII",
            "chrVIII", "chrIX", "chrX", "chrXI", "chrXII", "chrXIII",
            "chrXIV", "chrXV", "chrXVI")

# Functions ----

# Add additional columns
add_columns <- function(x){
  x$species <- sapply(strsplit(x[,1], "_"), "[", 2)

```

```

x$species <- factor(x$species,levels = refs)
x$chr <- sapply(strsplit(x[,1],"_"),"[[",1)
x$chr <- factor(x$chr,levels=allChr)
x <- x[order(x[,1],x[,2]),]
x$diff <- (x[,3] + x[,2])/2
x$rname <- factor(x$rname,levels=unique(x$rname))
df_temp <- split(x,x$rname)
add_rankcol <- function(z) {
  z$rank_chr <- 1:nrow(z)
  return(z)
}
df_temp <- lapply(df_temp,add_rankcol)
df_temp <- do.call(rbind, df_temp)
return(df_temp)
}

# plot the coverage across consecutive windows across chromosomes and species
# color gradient reflects per-window average mapping quality
plot_fun <- function(x) {
  ymax <- median(x[x$meandepth>0,"meandepth"])*4
  plot_list <- ggplot(x,mapping = aes(x=rank_chr,y=meandepth,color=meanmapq)) +
    geom_line(linewidth=0.2) +
    scale_color_gradient(low = "orange", high = "darkgreen", na.value = NA) +
    facet_grid(rows=vars(species),cols=vars(chr),scales = "free_x") +
    coord_cartesian(ylim = c(0, ymax))+
    theme_bw() +
    theme(
      panel.grid.major = element_blank(),
      panel.grid.minor = element_blank(),
      axis.line = element_line(colour = "black"),
      axis.title.x=element_blank(),
      axis.text.x=element_blank(),
      axis.ticks.x=element_blank(),
      text = element_text(size = 7,colour = "black"),
      axis.text.y = element_text(size = 7,colour = "black"),
      legend.position="bottom",
      plot.margin = margin(0,0,0,0, "cm")
    ) +
    ylab(
      paste0("Average coverage\n(",binsize/1000,"kb non overlapping windows)")
    ) +
    ggtitle(paste0("sample: ",unique(x$strain))) +
    guides(color = guide_colourbar(barwidth = 10, barheight = 0.25))
  return(plot_list)
}

# body ----

# list coverage files
cov_files <- list.files(path = paste0(BaseDir,"/cov/"),pattern = ".cov$")

# load all coverage files
df <- lapply(cov_files,function(x)

```



```
fread(cmd = paste0("cut -f1,2,3,7,9,10 ",
  BaseDir,"/cov/",x),data.table = FALSE))

# rename list elements
strains <- sapply(strsplit(cov_files,"\\."),"[" ,1)
names(df) <- strains

# rename columns of each data.frame
cols <- c("rname","startpos","endpos","meandepth","meanmapq","strain")
df <- lapply(df,setNames,cols)

# remove info of mitoDNA
df <- lapply(df, function(x) x[grepl("chrMT",x[,1],value = F,invert = T),])

# reshape the columns for the plots
df <- lapply(df,add_columns)

# ggplot2 and save the output to a single pdf

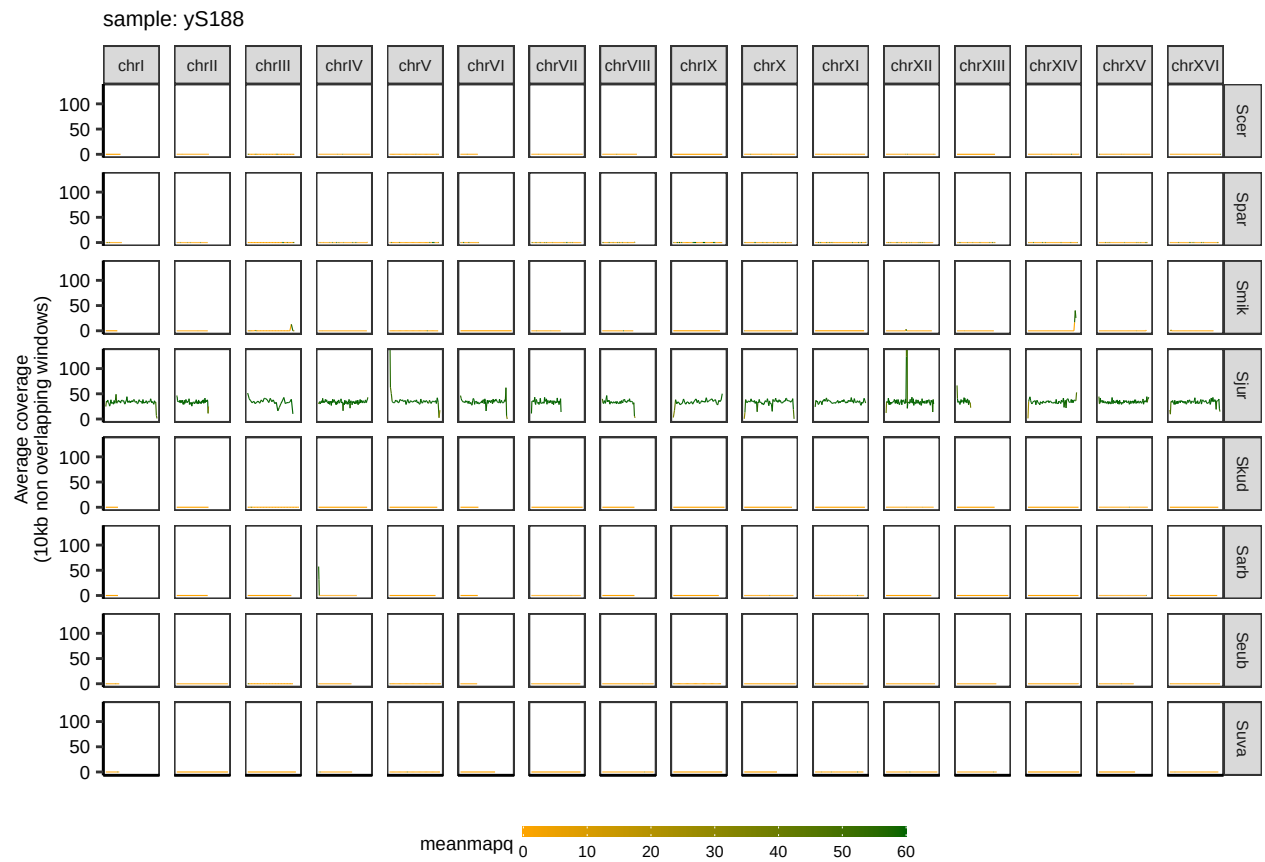
lapply(df, plot_fun)
```

```
## $SRR1119201
```



```
##
```

```
## $yS188
```



```
print("STEP3: completed!")
```

```
## [1] "STEP3: completed!"
```